

Algoritmos de aproximación aplicados a scheduling

List Scheduling y Longest Processing Time First

Docente: Héctor Eduardo Ugarte Rojas

Universidad Nacional de San Antonio Abad del Cusco
Carrera Profesional de Ingeniería Informática y de Sistemas

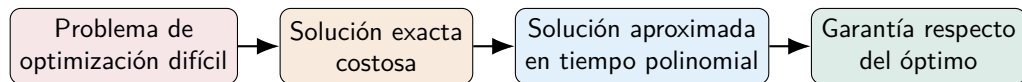
Semestre académico 2026-II

Propósito y resultados esperados

Propósito: comprender cómo obtener rápidamente soluciones factibles con garantía formal para un problema de optimización difícil.

Al finalizar la sesión, el estudiante podrá:

- 1 formular scheduling en máquinas idénticas para minimizar el makespan;
- 2 diferenciar solución óptima, aproximada y heurística;
- 3 calcular cotas inferiores y razones de aproximación;
- 4 ejecutar List Scheduling y LPT paso a paso;
- 5 explicar las garantías de List Scheduling y LPT;
- 6 comparar calidad de solución, tiempo y escalabilidad.



Pregunta guía

Si no podemos calcular el óptimo para una instancia grande, ¿cómo podemos juzgar la calidad de la solución obtenida?

Optimización, factibilidad y calidad

Un problema de optimización, como el del Scheduling, contiene:

- un conjunto de soluciones factibles $\mathcal{S}(I)$ para la instancia I ;
- una función objetivo $f(s)$;
- el objetivo de minimizar o maximizar f .

Para minimización:

$$\text{OPT}(I) = \min_{s \in \mathcal{S}(I)} f(s).$$

Tres preguntas distintas

¿La salida es factible? ¿Cuál es su valor objetivo? ¿Qué tan lejos puede estar del óptimo?

Una planificación es factible cuando cumple todas las restricciones del problema, el valor objetivo es el makespan en el caso del scheduling.

Aproximación frente a heurística

Algoritmo de aproximación

- tiempo polinomial;
- produce una solución factible;
- posee una garantía demostrada;
- se analiza para toda instancia válida.

Heurística

- busca soluciones de buena calidad;
- puede funcionar muy bien en la práctica;
- no necesariamente tiene garantía general;
- se evalúa principalmente de forma empírica.

No confundir

Que una heurística encuentre el óptimo en muchas pruebas no constituye una demostración de aproximación.

Una heurística es una regla, criterio o procedimiento práctico que utiliza información disponible, experiencia o conocimiento del problema para tomar decisiones y encontrar una solución razonable sin explorar exhaustivamente todas las posibilidades.

Razón de aproximación

Para un problema de minimización, un algoritmo es una ρ -aproximación si, para toda instancia I ,

$$\text{ALG}(I) \leq \rho \text{ OPT}(I), \quad \rho \geq 1.$$

Para maximización se exige:

$$\text{ALG}(I) \geq \frac{1}{\rho} \text{ OPT}(I).$$

Interpretación

Si un algoritmo es 2-aproximado para minimización, su valor nunca supera el doble del óptimo. No significa que normalmente sea dos veces peor.

Calidad observada y garantía teórica

Si conocemos el óptimo de una instancia pequeña, podemos calcular la calidad (razón de calidad observada):

$$r(I) = \frac{\text{ALG}(I)}{\text{OPT}(I)}$$

para minimización.

También puede reportarse la desviación relativa:

$$\delta(I) = \frac{\text{ALG}(I) - \text{OPT}(I)}{\text{OPT}(I)} \times 100 \%.$$

Ejemplo

Si $\text{ALG}(I) = 18$ y $\text{OPT}(I) = 15$, entonces $r(I) = 1.2$ y la desviación es 20 %.

Diferencia esencial

$r(I)$ describe una instancia; ρ es una garantía para todas las instancias.

Problema de scheduling: máquinas idénticas y makespan

Se tienen n trabajos independientes y m máquinas idénticas.

- makespan: tiempo total de ejecución o el tiempo finalización del último trabajo.
- El trabajo j tiene tiempo de procesamiento $p_j > 0$.
- Cada trabajo se asigna a exactamente una máquina.
- Cada máquina procesa un trabajo a la vez.
- No se interrumpe un trabajo una vez iniciado.
- El tiempo p_j es el mismo en cualquier máquina.
- El problema general de minimizar el makespan es computacionalmente difícil. Para instancias pequeñas podemos encontrar el óptimo mediante enumeración o Ramificar-Podar.

El objetivo es minimizar el makespan:

$$C_{\text{máx}} = \max_{1 \leq i \leq m} L_i,$$

donde L_i es la carga de la máquina i .

Formulación matemática

Sea $x_{ij} = 1$ si el trabajo j se asigna a la máquina i , y 0 en otro caso.

mín C

$$\sum_{i=1}^m x_{ij} = 1 \quad \forall j,$$

$$\sum_{j=1}^n p_j x_{ij} \leq C \quad \forall i,$$

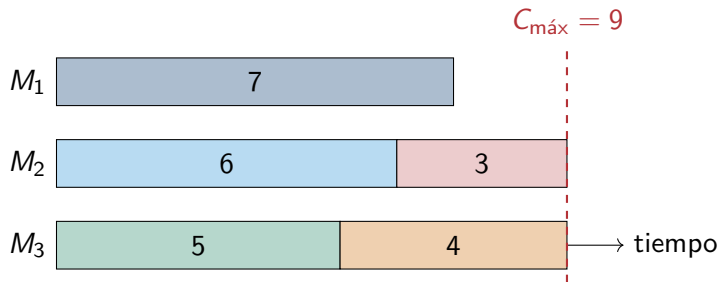
$$x_{ij} \in \{0, 1\}.$$

Interpretación

La primera restricción asigna cada trabajo una vez; la segunda obliga a que C sea al menos la carga de cada máquina.

Ejemplo de una planificación obtenida mediante LPT

Trabajos: $p = (7, 6, 5, 4, 3)$; máquinas: $m = 3$.



Cargas: $(7, 9, 9)$. La solución es factible y tiene makespan 9.

Dos cotas inferiores fundamentales

Toda planificación óptima satisface:

$$\text{OPT} \geq \frac{\sum_{j=1}^n p_j}{m}$$

porque la carga total debe distribuirse entre m máquinas.

También:

$$\text{OPT} \geq \max_j p_j$$

porque el trabajo más largo debe ejecutarse completo en alguna máquina.

Por tanto, Lower Bound (Cota inferior):

$$LB = \max \left\{ \frac{\sum_j p_j}{m}, \max_j p_j \right\} \leq \text{OPT}.$$

Cota inferior en el ejemplo

Para $p = (7, 6, 5, 4, 3)$ y $m = 3$:

$$\sum_j p_j = 25, \quad \frac{25}{3} \approx 8.33, \quad \max_j p_j = 7.$$

Como el makespan es entero en esta instancia:

$$LB = \left\lceil \frac{25}{3} \right\rceil = 9.$$

La planificación anterior tiene $C_{\max} = 9$ y la cota inferior también es 9.

Conclusión

Cuando una solución factible alcanza una cota inferior válida, la solución es óptima.

List Scheduling

Algoritmo de aproximación que procesa los trabajos en el orden recibido y asigna cada trabajo a una máquina de carga mínima.

Pseudocódigo

- ❶ Inicializar $L_i = 0$ para cada máquina i .
- ❷ Para cada trabajo j en el orden dado:
 - ❶ elegir una máquina i con carga mínima;
 - ❷ asignar j a i ;
 - ❸ actualizar $L_i \leftarrow L_i + p_j$.
- ❸ Retornar la asignación y $\max_i L_i$.

Los empates pueden resolverse con cualquier regla consistente.

Ejecución de List Scheduling

Orden recibido: (3, 7, 4, 6, 5); $m = 3$.

Trabajo	Carga antes	Máquina	Carga después	Vector de cargas
3	(0, 0, 0)	M_1	3	(3, 0, 0)
7	(3, 0, 0)	M_2	7	(3, 7, 0)
4	(3, 7, 0)	M_3	4	(3, 7, 4)
6	(3, 7, 4)	M_1	9	(9, 7, 4)
5	(9, 7, 4)	M_3	9	(9, 7, 9)

List Scheduling obtiene $C_{\max} = 9$, que en esta instancia es óptimo.

El orden puede cambiar la respuesta

Para los trabajos $p = (3, 3, 2, 2, 2)$ y $m = 2$:

Orden $(3, 2, 2, 3, 2)$

Cargas finales:

$(6, 6)$

Makespan: 6.

Orden $(3, 3, 2, 2, 2)$

Cargas finales:

$(7, 5)$

Makespan: 7.

Idea

List Scheduling es una familia de planificaciones: su calidad concreta depende del orden de la lista.

Garantía de List Scheduling

Para m máquinas idénticas:

$$C_{LS} \leq \left(2 - \frac{1}{m}\right) \text{OPT}.$$

En particular:

- $m = 2$: factor $3/2$;
- $m = 3$: factor $5/3$;
- cuando m crece: el factor se aproxima a 2.

Significado

Aunque el orden sea desfavorable, List Scheduling nunca supera el factor indicado respecto del óptimo.

Demostración de la garantía de List Scheduling

Sea j el último trabajo que termina en la máquina crítica y S_j su tiempo de inicio. Antes de asignar j , esa máquina tenía carga mínima, por lo que:

$$S_j \leq \frac{\sum_k p_k - p_j}{m}.$$

Entonces:

$$C_{LS} = S_j + p_j \leq \frac{\sum_k p_k}{m} + \left(1 - \frac{1}{m}\right) p_j.$$

Como $\sum_k p_k / m \leq \text{OPT}$ y $p_j \leq \text{OPT}$:

$$C_{LS} \leq \left(2 - \frac{1}{m}\right) \text{OPT}.$$

Complejidad de List Scheduling

Si en cada paso se busca linealmente la máquina menos cargada:

$$\mathcal{O}(nm).$$

Si las cargas se mantienen en una cola de prioridad mínima:

$$\mathcal{O}(n \log m).$$

Qué debe almacenarse

Cada entrada de la cola contiene la carga actual y el identificador de la máquina. Después de asignar un trabajo, se actualiza la carga y se reinserta la máquina.

Observación

La complejidad del algoritmo no incluye necesariamente el costo de leer o generar las instancias si esos pasos se miden por separado.

Longest Processing Time First, LPT

LPT modifica el orden antes de aplicar List Scheduling:

- 1 ordenar los trabajos por tiempo no creciente;
- 2 asignar cada trabajo a una máquina de carga mínima.

$$p_1 \geq p_2 \geq \cdots \geq p_n.$$

Intuición

Los trabajos largos son difíciles de acomodar al final. Al ubicarlos primero, los trabajos pequeños pueden rellenar mejor los desequilibrios restantes.

Ejecución de LPT

Trabajos recibidos: $(3, 7, 4, 6, 5)$; $m = 3$.

Orden LPT:

$(7, 6, 5, 4, 3)$.

Trabajo	Máquina	Carga después	Vector de cargas
7	M_1	7	$(7, 0, 0)$
6	M_2	6	$(7, 6, 0)$
5	M_3	5	$(7, 6, 5)$
4	M_3	9	$(7, 6, 9)$
3	M_2	9	$(7, 9, 9)$

LPT obtiene $C_{\max} = 9$.

LPT puede mejorar a List Scheduling

Para $p = (3, 1, 1, 1)$ y $m = 2$:

Lista desfavorable

Orden:

$(1, 1, 1, 3)$

Cargas finales:

$(2, 4)$

$C_{LS} = 4.$

Orden LPT

Orden:

$(3, 1, 1, 1)$

Cargas finales:

$(3, 3)$

$C_{LPT} = 3 = \text{OPT}.$

Pero LPT no siempre es óptimo

Ordenar de mayor a menor mejora la garantía de peor caso, pero existen instancias donde LPT no alcanza el óptimo.

Garantía de LPT

Para $m \geq 2$ máquinas idénticas:

$$C_{LPT} \leq \left(\frac{4}{3} - \frac{1}{3m} \right) \text{OPT}.$$

Ejemplos:

- $m = 2$: factor $7/6 \approx 1.167$;
- $m = 3$: factor $11/9 \approx 1.222$;
- cuando m crece: el factor se aproxima a $4/3$.

Comparación

LPT conserva la sencillez de List Scheduling y ofrece una garantía peor caso más fuerte gracias al ordenamiento previo.

Intuición detrás de la garantía LPT

Considere el último trabajo j de la máquina crítica.

- Si esa máquina recibe a lo sumo dos trabajos, la planificación resulta óptima.
- En un caso no óptimo relevante, j ocupa una posición suficientemente tardía en el orden.
- Por estar ordenados de mayor a menor, el trabajo crítico es relativamente pequeño:

$$p_j \leq \frac{\text{OPT}}{3}.$$

Sustituyendo esta información en el análisis de List Scheduling se obtiene la mejora hacia $4/3 - 1/(3m)$.

Alcance

La demostración formal completa requiere justificar cuidadosamente la propiedad del trabajo crítico.

Complejidad de LPT

Ordenar los n trabajos requiere:

$$\mathcal{O}(n \log n).$$

Asignar con una cola de prioridad para m cargas requiere:

$$\mathcal{O}(n \log m).$$

Por tanto:

$$\mathcal{O}(n \log n + n \log m) = \mathcal{O}(n \log n)$$

cuando $m \leq n$.

Costo adicional frente a List Scheduling

LPT invierte tiempo en ordenar para obtener una mejor garantía y, frecuentemente, una mejor solución práctica.

Comparación con una solución exacta

Para instancias pequeñas, puede calcularse OPT mediante enumeración o Ramificar-Podar.

Método	Ventaja	Limitación
Enumeración	Referencia simple y exacta	Hasta m^n asignaciones.
Ramificar-Podar	Exacto y puede podar mucho	Peor caso exponencial.
List Scheduling	Muy rápido y garantizado	Depende del orden.
LPT	Rápido y mejor garantía	No siempre óptimo.

La solución exacta permite medir la calidad observada de LS y LPT.

Métricas para la comparación experimental

Para cada instancia se recomienda registrar:

- makespan obtenido;
- cota inferior LB ;
- óptimo, cuando pueda calcularse;
- razón observada ALG / OPT ;
- desviación relativa;
- tiempo de ejecución;
- distribución de cargas;
- nodos explorados para el método exacto.

Comparación justa

Todos los algoritmos deben procesar exactamente la misma instancia y sus salidas deben validarse antes de medir calidad o tiempo.

Actividad en equipos

Trabajos: $p = (8, 7, 6, 5, 4, 3, 2)$; máquinas: $m = 3$.

- 1 Calcule la cota inferior LB .
- 2 Ejecute List Scheduling con el orden $(3, 8, 2, 7, 4, 6, 5)$.
- 3 Ejecute LPT.
- 4 Compare los makespan obtenidos.
- 5 Determine si alguna solución alcanza la cota inferior.
- 6 Proponga una razón por la que el orden afecta a LS.

Tiempo sugerido

Cinco minutos de trabajo y tres minutos para contrastar respuestas.

Cierre: ideas esenciales

- 1 Una solución aproximada debe ser factible y puede tener garantía formal.
- 2 Las cotas inferiores permiten juzgar calidad sin conocer siempre el óptimo.
- 3 List Scheduling asigna al recurso menos cargado y depende del orden.
- 4 LPT ordena trabajos de mayor a menor y mejora la garantía.
- 5 La evaluación debe considerar calidad, tiempo, escalabilidad y supuestos.

Siguiente sesión

Aproximación avanzada: redondeo, Vertex Cover, PTAS y FPTAS.

Responda individualmente:

- 1 ¿Qué diferencia existe entre razón observada y garantía de aproximación?
- 2 ¿Cuáles son las dos cotas inferiores básicas para el makespan?
- 3 ¿Por qué el orden de trabajos afecta a List Scheduling?
- 4 ¿Qué modificación introduce LPT?
- 5 ¿Por qué una solución que alcanza una cota inferior es óptima?

Referencias y recursos

-  T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, 4.^a ed. MIT Press, 2022.
-  V. V. Vazirani. *Approximation Algorithms*. Springer, 2001.
-  D. P. Williamson y D. B. Shmoys. *The Design of Approximation Algorithms*. Cambridge University Press, 2011.
-  R. L. Graham. Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics*, 17(2), 1969.
-  F. Della Croce y R. Scatamacchia. *Longest Processing Time rule for identical parallel machines revisited*. 2018. <https://arxiv.org/abs/1801.05489>